

# Final Report for Summer Research 2024

Baoduo Lv<sup>1</sup> and Mentor: Tian Gan<sup>1</sup>

<sup>1</sup>Department of Computer Science and Engineering, Tandon School of Engineering, New York University

\*Email: bl3274@nyu.edu

## ABSTRACT

This project seeks to explore the geographic accessibility of doctors by developing multi-layer networks based on various geographic factors, including distance and public transit. Leveraging proficiency in Python and network science, the research involves several key steps. Initially, a literature review on transportation networks is conducted to understand geographic-related tools. This is followed by data collection, specifically acquiring GTFS data for New York City's subways and buses, and excluding trips from midnight to 5 am. Utilizing the Networkx library, a graph of the public transport network is created, where nodes represent subway and bus stops, and edges are defined by average travel time and the number of vehicle trips per route. A two-layer transport network is then developed by merging the subway and bus networks. Subsequently, an accessibility network of doctors is constructed based on the transport network. The public transport network is compared with the doctor network, culminating in a comprehensive analysis.

## Introduction

Access to healthcare is becoming a significant issue for marginalized groups, including for many elderly individuals and new immigrants. In this project, we aim to explore the accessibility of doctors more thoroughly and provide insights to support better policy-making. In this project, two types of public transport were considered: subway and bus. The General Transit Feed Specification (GTFS) data for both subways and buses, including station locations and vehicle trips, were collected from the MTA website [1]. After preprocessing, which included removing vehicle trips occurring after 6 pm and before 9 am, as well as merging nearby stations, the transport network included 446 subway stations and 5,527 bus stations. Additionally, specialty doctors were not considered, and 436 family doctors were incorporated into the doctor network.

## 1 Methods

### 1.1 Data Collection

The data for both subway and bus networks was obtained from the Google Transit Feed Specification (GTFS) datasets. These datasets were sourced from respective directories containing `filtered_trips.txt`, `stop_times.txt`, and `stops.txt` files for multiple transport modes. Data was loaded into a Python environment for further processing and analysis.

### 1.2 Data Processing

- Data Merging:** For each transport mode, the `trips`, `stop_times`, and `stops` datasets were merged using `trip_id` and `stop_id` to create a unified dataset. This merging facilitated the integration of trip sequences and geographical stop locations into a single data structure.
- Station Normalization:** Stop IDs were standardized to their parent stations when applicable. This normalization involved replacing `stop_id` with `parent_station` to ensure consistent station identification across datasets.
- Data Concatenation:** For the bus network, data from multiple directories was concatenated to form a complete dataset representing the entire network. This involved combining the lists of trips, stop times, and stops into a single comprehensive dataset.

### 1.3 Network Construction

- Graph Creation:** The transportation networks were modeled as directed graphs using the NetworkX library. In these graphs, each node represents a station, and each directed edge represents a sequential connection between stations as indicated by the trip data.
- Adjacency Matrix Construction:**

- (a) **Functionality:** The adjacency matrix was constructed to capture the frequency of trips and travel times between stations.

(b) **Pseudo Code for Building the Adjacency Matrix:**

```
Function build_matrix(DataFrame stop_details):
    Initialize trip_count as a DataFrame of zeros with stop_1st as
    both index and columns
    Initialize travel_time as a DataFrame of zeros with stop_1st as
    both index and columns

    For i from 0 to length of stop_details - 1:
        curr_seq <- stop_sequence at stop_details[i]
        curr_id <- stop_id at stop_details[i]
        next_seq <- stop_sequence at stop_details[i+1]
        next_id <- stop_id at stop_details[i+1]

        If curr_seq + 1 equals next_seq:
            Increment trip_count[curr_id, next_id] by 1
            travel_time[curr_id, next_id] <- (arrival_time at
            stop_details[i+1] - departure_time at stop_details[i])
            in minutes

    Return trip_count, travel_time
```

3. **Node Merging for nearby stations:** To address closely located stops, a clustering method was employed to merge nodes within a 200-meter threshold using the Haversine distance. This process grouped nearby stops to simplify the network structure without compromising geographic accuracy.

(a) **Pseudo Code for Node Merging:**

```
Function merge_close_nodes(Graph G, Number threshold):
    positions <- Get node attributes from G with key 'pos'
    nodes <- List of nodes in G
    clusters <- Empty list for storing clusters
    visited <- Empty set for tracking visited nodes

    For each node in nodes:
        If node is not in visited:
            cluster <- Set containing node
            queue <- List containing node
            Add node to visited
            While queue is not empty:
                current <- Remove the first element from queue
                current_pos <- Position of current in G
                For each neighbor in nodes:
                    If neighbor is not in visited:
                        neighbor_pos <- Position of neighbor in G
                        If neighbor is within threshold distance of all
                        nodes in cluster:
                            Add neighbor to visited
                            Append neighbor to queue
                            Add neighbor to cluster
            Append cluster to clusters
```

```

For each cluster in clusters:
    For each node in cluster:
        str_val <- Concatenate cluster nodes' name into a string
        stops_mapping[node] <- str_val

Return clusters

```

4. **Network Visualization:** The final transport network for bus and subway can be visualized using the two matrices `trip_count` and `travel_time` and the `Networkx` library.

(a) **Pseudo Code for Visualization:**

```

Function build_network(List clusters, DataFrame trip_count, DataFrame
travel_time, String title):
    Initialize new directed graph new_G
    Get node attributes 'name' from global graph G
    Get list of nodes from global graph G

    For each cluster in clusters:
        Calculate centroid of cluster by averaging latitude and longitude
        of nodes
        Combine names of all nodes in the cluster into a single string
        Create a new node with the combined name and centroid position
        in new_G

    For each node i in trip_count index:
        For each node j in trip_count columns:
            If there is at least one trip from node i to node j:
                Get routes of node i and node j from new_G
                Calculate intersection of routes_i and routes_j to find
                common routes
                If common routes exist:
                    Add edge from i to j in new_G with attributes: number
                    of trips, travel time, and common routes

    Remove self-loop edges from new_G

    Get position attributes from new_G
    Get name attributes from new_G
    Create edge labels with vehicle trips and travel time for each edge in
    new_G

    Plot the graph:
        Set figure size
        Draw new_G with specified attributes
        Add edge labels to the graph
        Show the graph

    Save the graph as a pickle file with the specified title

    Return new_G

```

## 1.4 Merging Subway and Bus Networks

1. **Graph Creation:** A new directed graph,  $G_{\text{merged}}$ , is created to represent the integrated transport network combining both subway and bus networks.

2. **Network Integration:**

- (a) Nodes and edges from both networks are added to  $G_{\text{merged}}$  with prefixes "subway\_" and "bus\_" to distinguish between the different transport modes.
- (b) Transfer points between the subway and bus nodes are evaluated based on a distance threshold. These points represent potential walking transfers between close-proximity stations of different networks.

- (c) **Pseudo Code for Merging Networks:**

```
Function merge_subway_bus(Graph subway_network, Graph bus_network,
Number transfer_distance_threshold, Number walking_speed=1.47):
    Initialize G_merged as a new directed graph

    For each node, data in subway_network:
        Add node with prefix "subway_" to G_merged with original data
    For each edge in subway_network:
        Add edge with prefixed nodes to G_merged with original data

    For each node, data in bus_network:
        Add node with prefix "bus_" to G_merged with original data
    For each edge in bus_network:
        Add edge with prefixed nodes to G_merged with original data

    For each bus_node in bus_network.nodes:
        For each subway_node in subway_network.nodes:
            Calculate distance between bus_node and subway_node
            If distance <= transfer_distance_threshold:
                Calculate transfer time based on walking speed
                Add transfer edges between bus and subway nodes

    Return G_merged
```

3. **Visualization:** The merged network is visualized to illustrate the connectivity between the subway and bus layers, highlighting transfer points and their respective travel times based on walking distance.

## 1.5 Shortest Path Computation

1. **Algorithm Customization:** The shortest path calculation is adapted to account for multi-modal transfers, varying transfer times, and additional delays at stops using a modified Dijkstra algorithm. This approach allows for more realistic modeling of travel times in urban transport networks.

2. **Pseudo Code for Modified Dijkstra Algorithm:**

```
Function compute_shortest_path(Graph G, Node source, Node target,
Number transfer_time=6, Number stop_time=1):

    Function dijkstra_with_transfers(Graph G, Node source, Node target):
        Initialize distances as a dictionary with all nodes set to infinity
        Set distance to source node as 0
        Initialize priority queue (pq) with (distance=0, source,
        edge routes (None for now))
        Initialize previous_nodes as a dictionary to track the shortest path
```

```

While priority queue is not empty:
    Pop the node with the smallest distance from pq
    If the popped distance is greater than the recorded distance
    for the node, continue to the next iteration

    For each neighbor of the current node:
        Set weight as the travel time from current_node to neighbor
        Get the set of routes for the edge from current_node to neighbor

        If there is a previous edge route and it does not intersect with
        the set of edges (a transfer needed):
            If the edge is a transfer between two layers:
                Add transfer time and noise (5-10 minutes) to the weight
            Else:
                Add transfer time and noise (0-5 minutes) to the weight

        Calculate new_distance as the sum of current_distance, weight,
        and stop_time

        If new_distance is less than the recorded distance for neighbor:
            Update the distance for neighbor
            Set previous_nodes[neighbor] to current_node
            Push (new_distance, neighbor, edge_routes) onto the priority
            queue

Initialize path as an empty list
Set current_node to target
While current_node is not None:
    Append current_node to path
    Set current_node to the previous node in previous_nodes
Reverse the path to get the correct order

Return the path and the total travel time (distance to target)

Call dijkstra_with_transfers with G, source, and target
Return the shortest path and total travel time

```

3. **Implementation Details:** The algorithm is implemented to specifically handle transitions between different modes of transport within the network, such as from bus routes to subway lines, incorporating realistic scenarios of waiting and transfer times.

## 1.6 Doctor Accessibility Computation

The subsection is focused on analyzing the accessibility of doctors for different zip code residents using a multi-layer transport network (comprising subway and bus networks). The primary tasks include determining the nearest station for both zip code centroids and doctors, calculating the shortest path between these points, and recording the total travel time.

### 1. Nearest Station Identification:

- (a) **Zip Codes:** The function `get_nearest_stop_zipcode` is used to determine the nearest transport station for each zip code centroid. It calculates the distance between the centroid and various transport stations using the Haversine formula and returns the nearest station within a specified distance limit.

- (b) **Doctors:** Similarly, `get_nearest_stop_doctor` identifies the closest transport station to each doctor's location. The nearest station is determined by the shortest distance calculated using the Haversine formula, provided it is within a given distance limit.

## 2. Path Calculation:

- (a) The function `compute_path_from_zipcode_to_doctor` calculates the shortest path between a zip code centroid and a doctor's location using the multi-layer transport network. The calculation includes walking time from the centroid to the nearest station, the transit time on the network, and the walking time from the station to the doctor's location.

## 3. Accessibility Analysis:

- (a) The function `access_by_subway_bus` evaluates the accessibility of every doctor based on the transport network. It determines which zip codes can reach which doctors within a specified time limit, considering both walking and transit times. The results are returned as a mapping of doctors to accessible zip codes.

## 4. Pseudo Code for Accessibility Functions:

```
Function get_nearest_stop_zipcode(Map centroids_pos, Map station_info, Number distance_limit):
```

```
    Initialize nearby_stop as an empty dictionary
    For each zipcode, centroid_pos in centroids_pos:
        Initialize curr_best as distance_limit
        For each stop_id, stop_pos in station_info:
            Calculate the distance between centroid_pos and stop_pos
            If distance is less than curr_best:
                Update nearby_stop with stop_id and stop_pos
                Set curr_best to the current distance
    Return nearby_stop
```

```
Function get_nearest_stop_doctor(Map doctors_pos, Map station_info, Number distance_limit):
```

```
    Initialize nearby_stop as an empty dictionary
    For each doctor_id, doctor_pos in doctors_pos:
        Initialize curr_best as distance_limit
        For each stop_id, stop_pos in station_info:
            Calculate distance between doctor_pos and stop_pos
            If distance is less than curr_best:
                Update nearby_stop with stop_id and stop_pos
                Set curr_best to the current distance
    Return nearby_stop
```

```
Function compute_path_from_zipcode_to_doctor(Graph network, Tuple zipcode_info, Tuple doctor_info, List zipcode_nearest_stations, List doctor_nearest_stations, Number time_limit, Number walking_speed=1.47):
```

```
    Initialize curr_best as (None, time_limit)
    For each stop_z in zipcode_nearest_stations:
        For each stop_d in doctor_nearest_stations:
            Calculate path and transit_time using shortest path algorithm
            Calculate time_to_station and time_to_doctor based on walking_speed
            Calculate total_time as sum of time_to_station, transit_time, and time_to_doctor
            If total_time is within time_limit:
                Update curr_best with new path and total_time
    Return curr_best if a valid path exists, otherwise return None
```

```

Function access_by_subway_bus(Graph transport_network, Map accessible_zipcodes,
Map accessible_doctors, Number time_limit, Number walking_speed=1.47):
    Initialize zipcodes_per_doctor as an empty dictionary
    For each doctor_info, nearest_stops_d in accessible_doctors:
        For each zipcode_info, nearby_stops_z in accessible_zipcodes:
            Calculate distance between zipcode_info and doctor_info
            If distance is less than 1000 meters:
                Calculate walking_time and add to zipcodes_per_doctor
            Else:
                Compute path using compute_path_from_zipcode_to_doctor
                If a valid path exists, add to zipcodes_per_doctor
        Print completion message for the doctor
    Return zipcodes_per_doctor

```

## 1.7 Doctor Accessibility Network Construction

The following part includes computing the number of shared zip codes between each pair of doctors based on the previously calculated accessibility. The primary tasks involve determining which zip codes are accessible to each doctor under a specific time limit of 28.4 minutes, and then identifying how many zip codes are shared between every pair of doctors.

### 1. Matrix Construction:

- (a) The function `doctor_matrix` computes the shared zip codes between each pair of doctors by first creating a matrix where each row represents a doctor and each column represents a zip code. The matrix is populated based on whether a doctor has access to residents of a particular zip code.
- (b) A doctor-to-doctor matrix is then generated using matrix multiplication, where each entry represents the number of shared zip codes between two doctors.

### 2. Network Visualization:

- (a) The function `plot_doctor_network` visualizes the network of doctors, where nodes represent doctors and edges represent the number of shared zip codes. Nodes are positioned using geolocation data, and the network is overlaid on a map to illustrate geographical distribution.

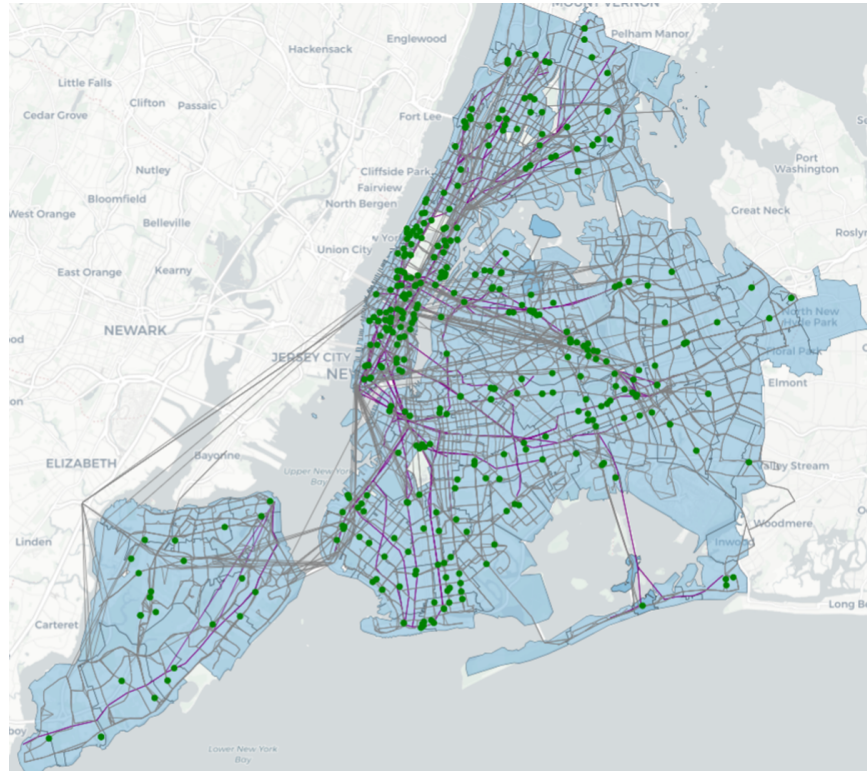
### 3. Pseudo Code for Doctor-Shared Zip Code Analysis:

```

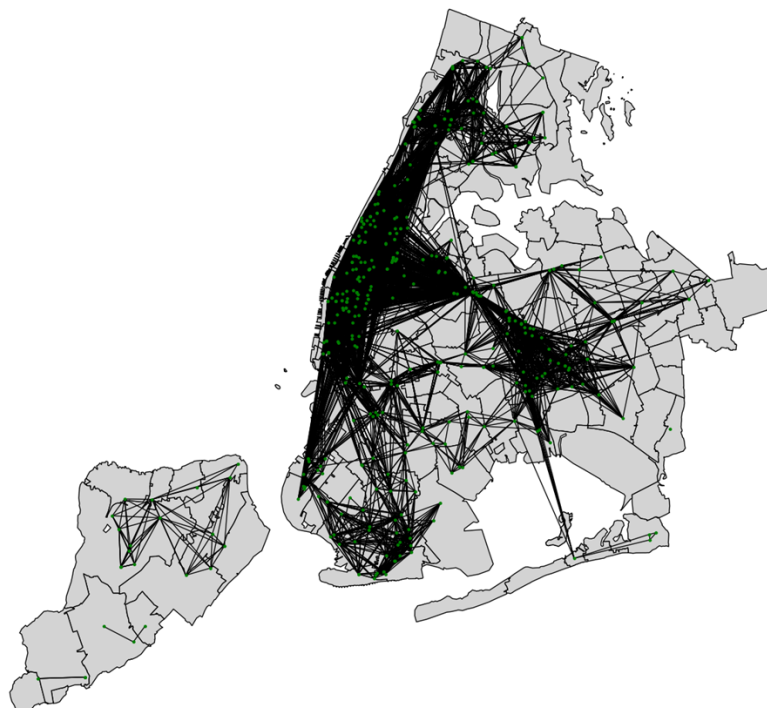
Function doctor_matrix(DataFrame data):
    Initialize doctor_zipcode matrix with zeros (rows: doctors, columns: zipcodes)
    For each row in data:
        Set doctor_zipcode[Doctor, Zipcode] to 1 if accessible
    Compute doctor_doctor matrix by multiplying doctor_zipcode with its transpose
    Return doctor_zipcode, doctor_doctor

Function plot_doctor_network(DataFrame matrix_df):
    Initialize graph G
    Add nodes to G from doctor list in matrix_df
    Add edges to G based on shared zipcodes (weight > 0)
    Calculate centrality measures (degree, betweenness)
    Plot network on a geographical map
    Return G

```

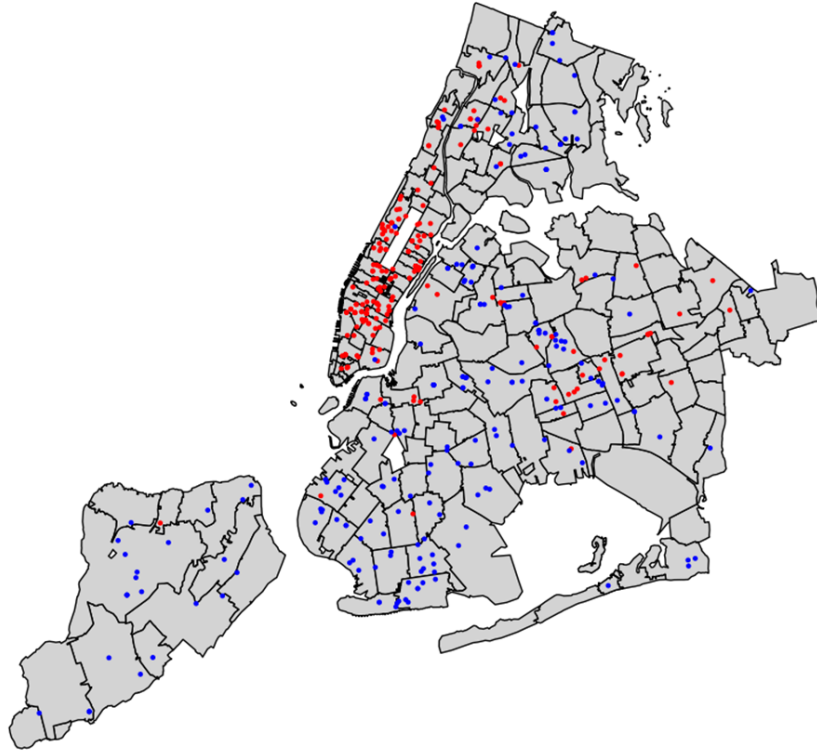


**Figure 1.** Map of NYC shows subway routes (purple lines), bus routes (grey lines), and doctor distribution (green dots).



**Figure 2.** Accessibility network of 436 family doctors;





**Figure 3.** Distribution of family doctors with high competition (red dots) and family doctors with low competition (blue dots) in NYC.

## Results

After executing the entire pipeline of algorithms, the transport network and the doctor network were constructed, allowing for subsequent observations and analysis.

As shown in Fig. 1, many doctors locate their clinics near subway stations, and the two-layer transport network effectively facilitates transportation to most doctors in NYC.

Fig. 2 is constructed with a travel time limit of 28.4 minutes<sup>7</sup>. In the graph, a link between two doctors indicates that they share patients from one or more zip codes, establishing a connection between them. Within the time period considered, most doctors are linked to at least one other doctor, with Manhattan being the densest area. This indicates that doctors in Manhattan have more connections with each other compared to other areas.

Fig. 3 illustrates how doctor nodes are distributed according to their number of connections to other doctors. Red nodes (high-degree nodes) signify doctors with a high number of connections, while blue nodes (low-degree nodes) indicate those with fewer connections. As depicted, areas with high populations like Manhattan tend to have a greater concentration of high-degree nodes, whereas low-degree nodes are spread more evenly across the entire city.

## References

1. Metropolitan Transportation Authority. *MTA Developer Resources*. Retrieved from <https://new.mta.info/developers>.
2. Yen, W. (2013). *How Long and How Far Do Adults Travel and Will Adults Travel for Primary Care?* Washington State Office of Financial Management.